

Game Playing: Adversarial Search

State of the art

- **Chess:**
 - Deep Blue beat Gary Kasparov in 1997
 - Garry Kasparov vs. Deep Junior (Feb 2003): tie!
 - Kasparov vs. X3D Fritz (November 2003): tie!
- **Checkers:** Chinook is the world champion
- **Checkers:** has been solved exactly – it's a draw!
- **Go:** Computer players are decent, at best
- **Bridge:** “Expert” computer players exist, but no world champions yet
- **Poker:** CPRG regularly beats human experts
 - <https://poker.cs.ualberta.ca/>
- Check out: <http://www.cs.ualberta.ca/~games/>

Why study games

- Fun
- Clear criteria for success
- Offer an opportunity to study problems involving {hostile, adversarial, competing} agents.
- Interesting, hard problems which require minimal “initial structure”
- Games often define very large search spaces
 - Checkers: 5×10^{20}
 - Chess 10^{120} nodes

Typical Case (perfect games)

- 2-player game
- Players alternate moves
- Zero-sum-- one player's loss is the other's gain.
- Perfect information -- both players have access to complete information about the state of the game. No information is hidden from either player.
- No chance (e.g., using dice) involved
- Clear rules for legal moves (no uncertain position transition involved)
- Well-defined outcomes (Win/Loss/Draw)
- Examples: Tic-Tac-Toe, Checkers, Chess, Go, Nim, Othello
- Not: Bridge, Solitaire, Backgammon, ...

How to play a game

- A way to play such a game is to:
 - Consider all the legal moves you can make.
 - Each move leads to a new board configuration (position).
 - **Evaluate** each resulting position and determine which is best
 - Make that move.
 - **Wait for your opponent to move and repeat**
- Key problems are:
 - Representing the “board”
 - Generating all legal next boards
 - **Evaluating a position**
 - **Look ahead**

Game Trees

- Problem spaces for typical games represented as trees.
- Root node represents the “board” configuration at which a decision must be made as to what is the **best single move to make next**. (not necessarily the initial configuration)
- **Evaluator function** rates a board position. $f(\text{board})$ (a real number).
- Arcs represent the possible legal moves for a player (no costs associates to arcs)
- **Terminal nodes** represent end-game configurations (the result must be one of “win”, “lose”, and “draw”, possibly with numerical payoff)

- If it is my turn to move, then the root is labeled a "**MAX**" node; otherwise it is labeled a "**MIN**" node indicating my opponent's turn.
- Each level of the tree has nodes that are all MAX or all MIN; nodes at level i are of the opposite kind from those at level $i+1$
- Complete game tree: includes all configurations that can be generated from the root by legal moves (all leaves are terminal nodes)

Evaluation Function

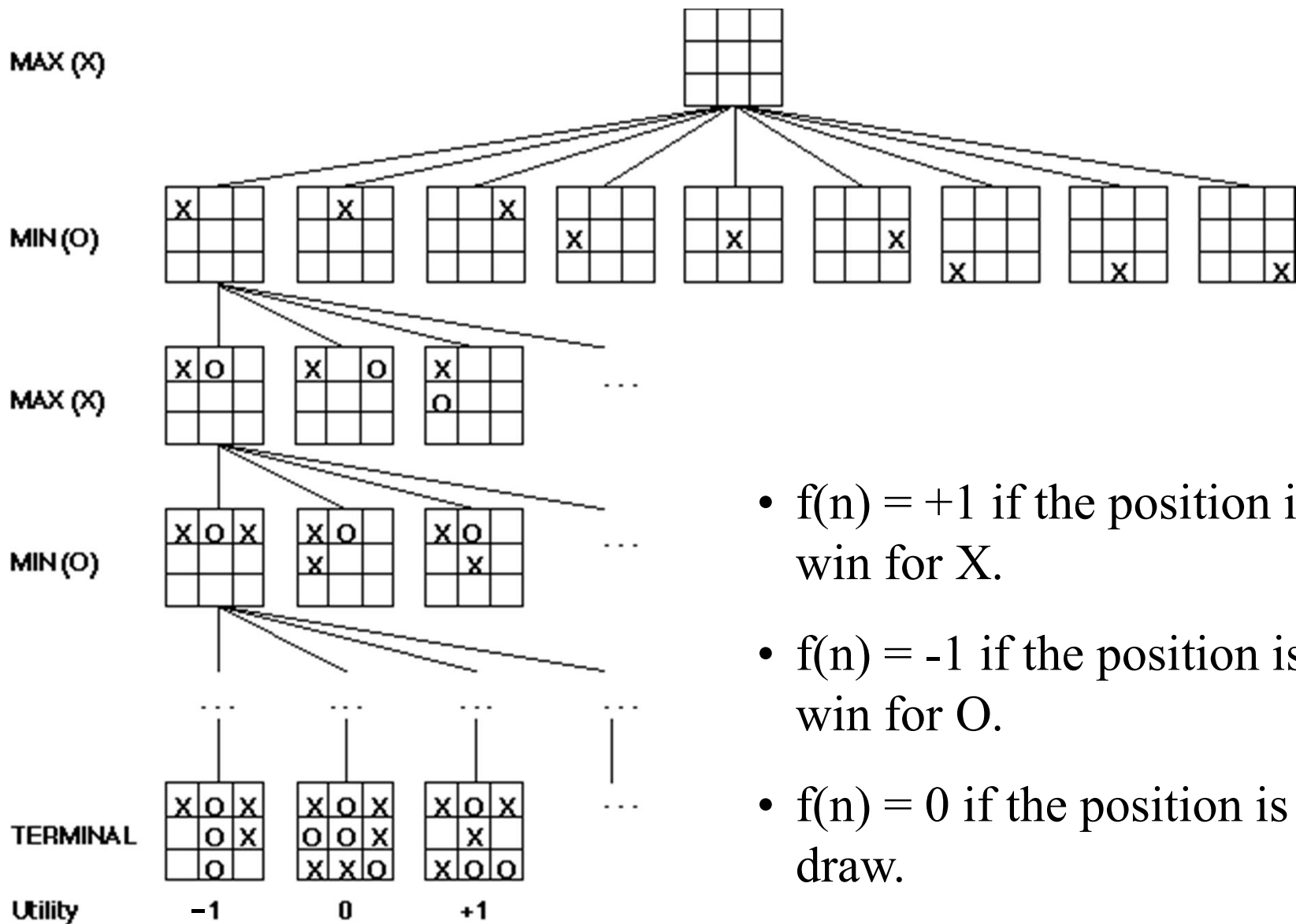
- **Evaluation function** or **static evaluator** is used to evaluate the "goodness" of a game position.
 - Contrast with heuristic search where the evaluation function was a non-negative estimate of the cost from the start node to a goal and passing through the given node.
- The zero-sum assumption allows us to use a single evaluation function to describe the goodness of a board with respect to both players.
 - $f(n) > 0$: position n good for me and bad for you.
 - $f(n) < 0$: position n bad for me and good for you
 - $f(n)$ near 0 : position n is a neutral position.
 - $f(n) \gg 0$: win for me.
 - $f(n) \ll 0$: win for you..

- Evaluation function is a heuristic function, and it is where the domain experts' knowledge resides.
- Example of an Evaluation Function for Tic-Tac-Toe:

$$f(n) = [\text{\# of 3-lengths open for me}] - [\text{\# of 3-lengths open for you}]$$
 where a 3-length is a complete row, column, or diagonal.
- Alan Turing's function for chess
 - $f(n) = w(n)/b(n)$ where $w(n)$ = sum of the point value of white's pieces and $b(n)$ is sum for black.
- Most evaluation functions are specified as a weighted sum of position features:

$$f(n) = w_1 * \text{feat}_1(n) + w_2 * \text{feat}_2(n) + \dots + w_n * \text{feat}_k(n)$$
- Example features for chess are piece count, piece placement, squares controlled, etc.
- Deep Blue has about 6,000 features in its evaluation function.

An example (partial) game tree for Tic-Tac-Toe



- $f(n) = +1$ if the position is a win for X.
- $f(n) = -1$ if the position is a win for O.
- $f(n) = 0$ if the position is a draw.

Minimax Rule

- Goal of game tree search: to determine **one move** for Max player that **maximizes** the **guaranteed payoff** for a given game tree for MAX

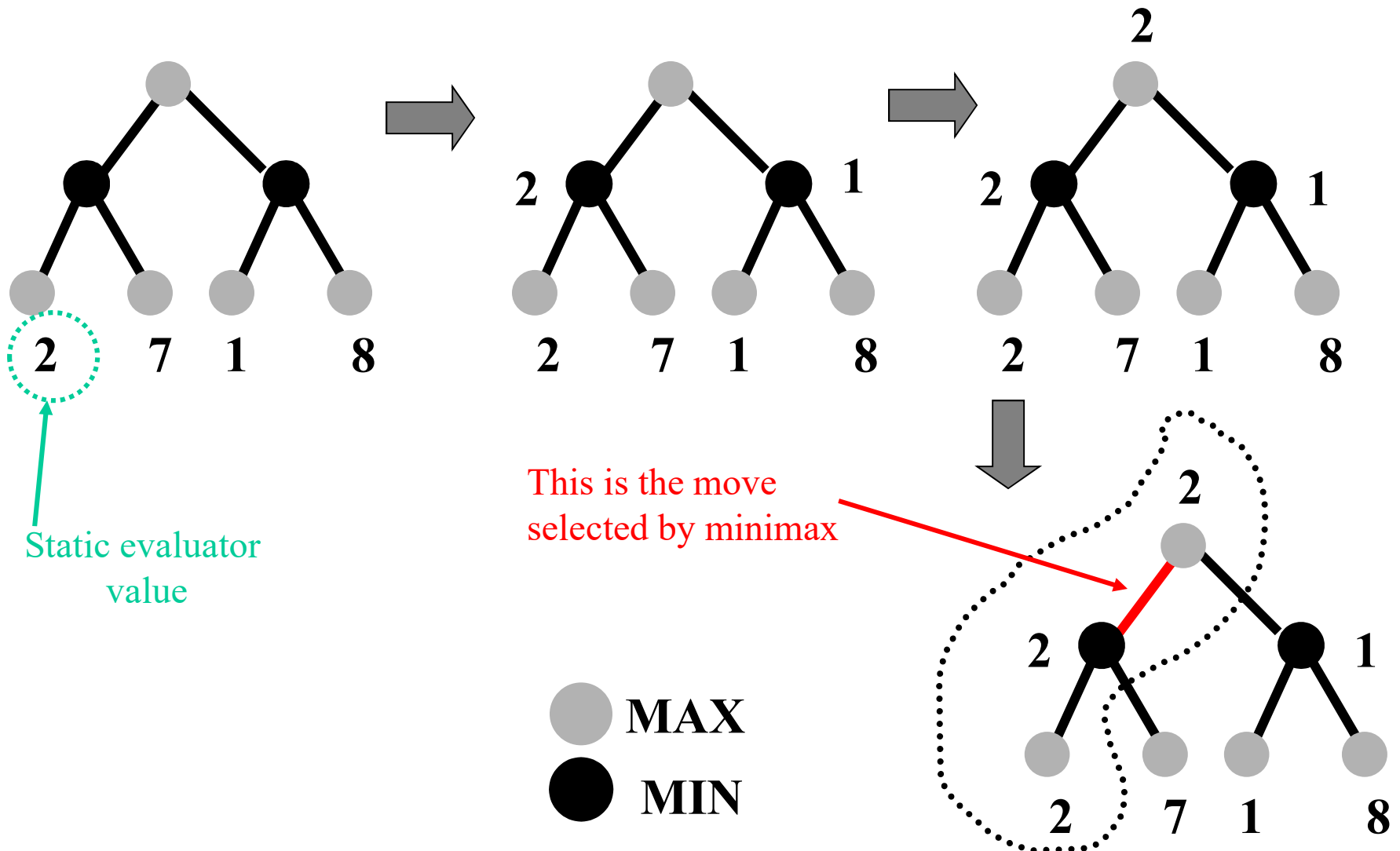
Regardless of the moves the MIN will take

- The value of each node (Max and MIN) is determined by (back up from) the values of its children
- **MAX plays the worst case scenario:**
Always assume MIN to take moves to maximize his pay-off (i.e., to minimize the pay-off of MAX)
- For a MAX node, the backed up value is the **maximum** of the values associated with its children
- For a MIN node, the backed up value is the **minimum** of the values associated with its children

Minimax procedure

- Create start node as a MAX node with current board configuration
- Expand nodes down to some depth (i.e., ply) of lookahead in the game.
- Apply the evaluation function at each of the leaf nodes
- Obtain the “**back up**” values for each of the non-leaf nodes from its children by Minimax rule until a value is computed for the root node.
- Pick the operator associated with the child node whose backed up value determined the value at the root as the move for MAX

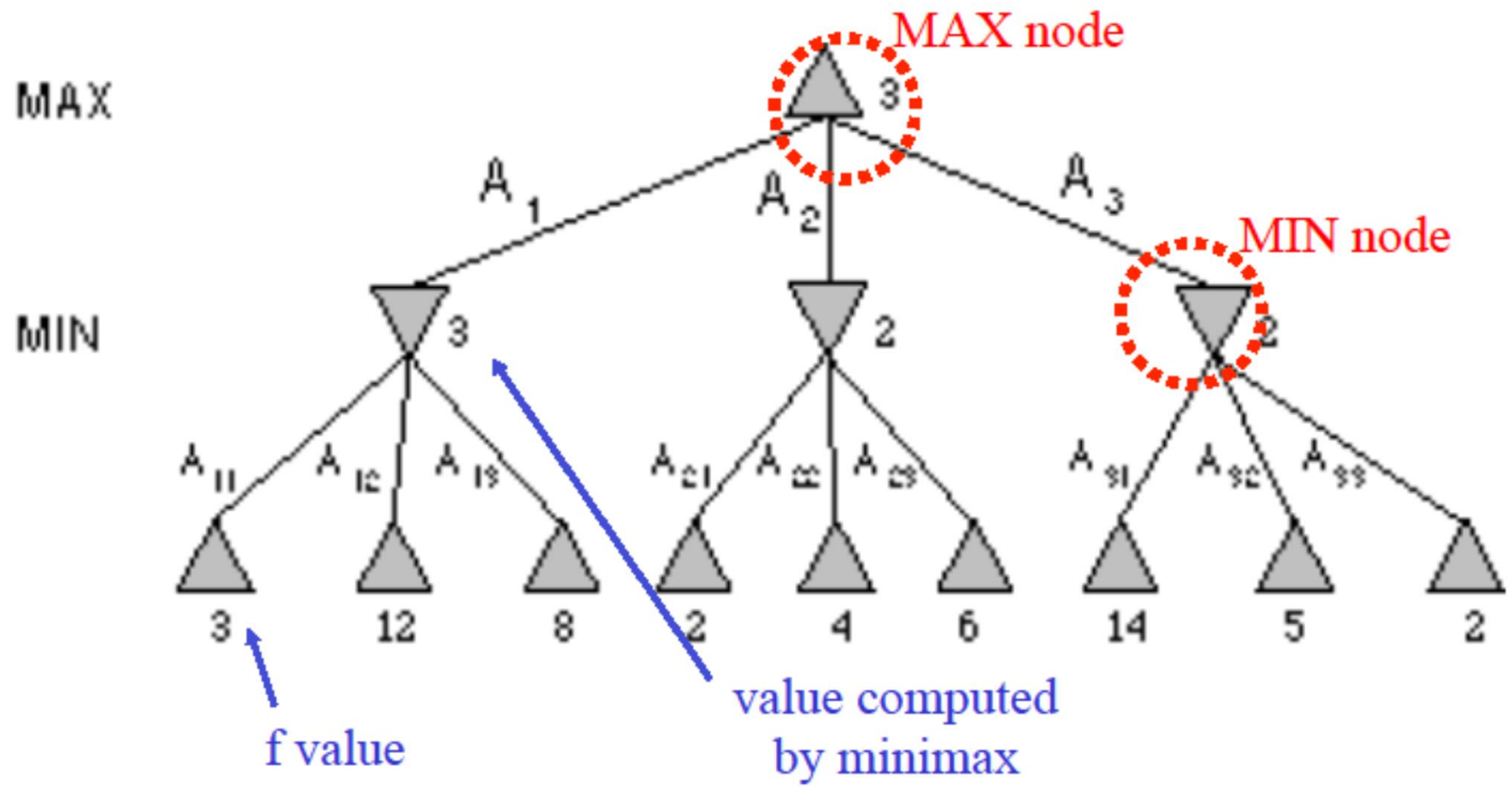
Minimax Search



Comments on Minimax search

- The search is **depth-first** with the given depth (ply) as the limit
 - Time complexity: $O(b^d)$
 - Linear space complexity
- **Performance** depends on
 - Quality of evaluation functions (domain knowledge)
 - Depth of the search (computer power and search algorithm)
- Different from ordinary state space search
 - Not to search for a solution but for one move only
 - No cost is associated with each arc
 - MAX does not know how MIN is going to counter each of his moves
- Minimax rule is a basis for other game tree search algorithms

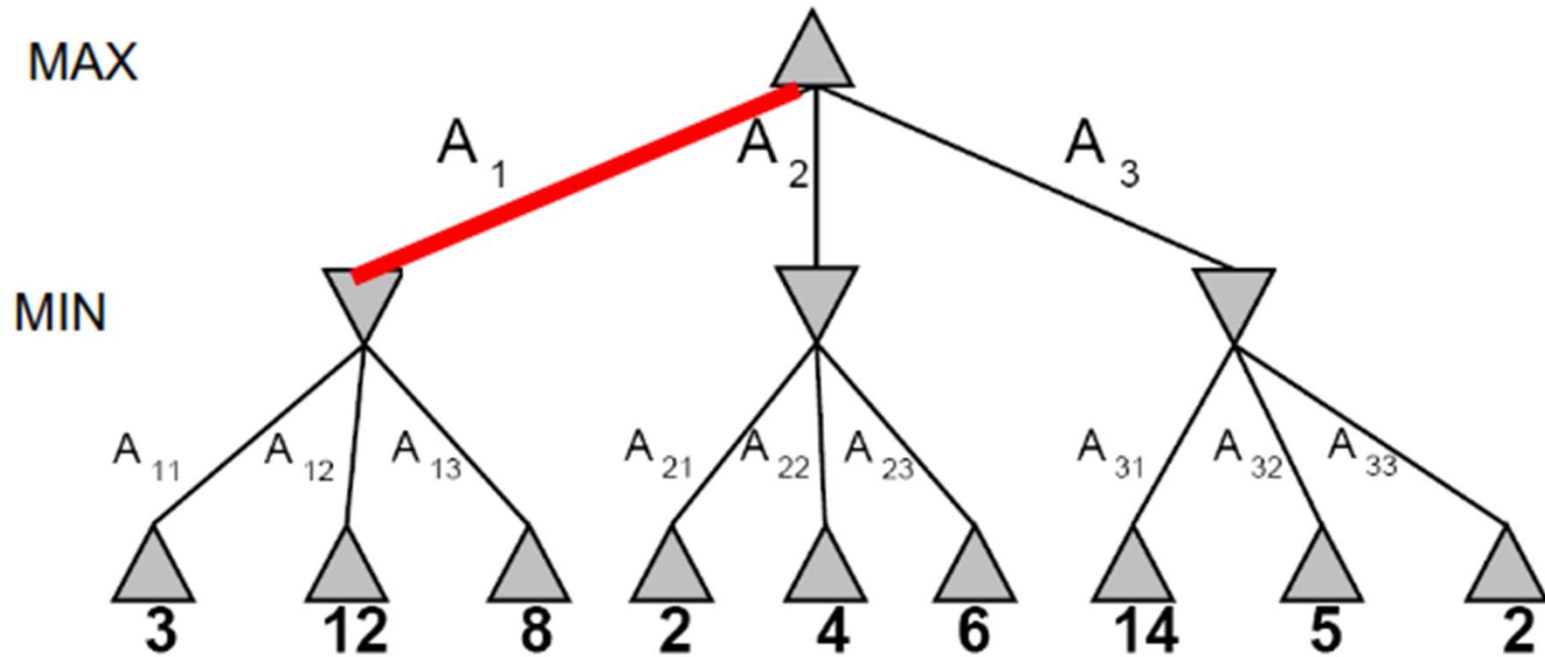
Minimax Tree



The minimax algorithm

- Perfect play for deterministic environments with perfect information
- **Basic idea:** choose move with highest minimax value
= best achievable payoff against best play
- **Algorithm:**
 1. Generate game tree completely
 2. Determine utility of each terminal state
 3. Propagate the utility values upward in the tree by applying MIN and MAX operators on the nodes in the current level
 4. At the root node use minimax decision to select the move with the max (of the min) utility value
- Steps 2 and 3 in the algorithm assume that the opponent will play perfectly.

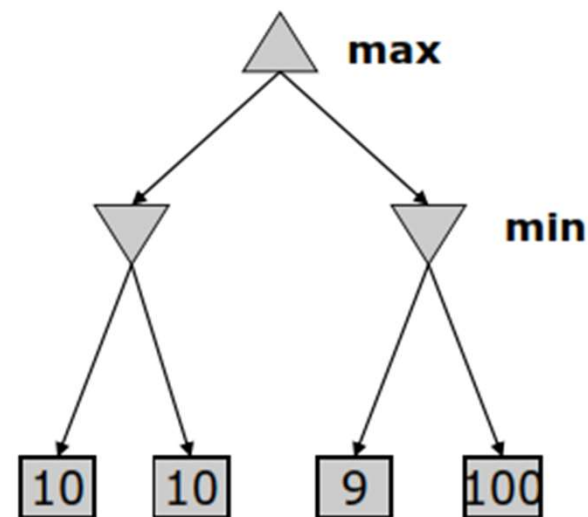
Minimax Example



The Minimax Algorithm

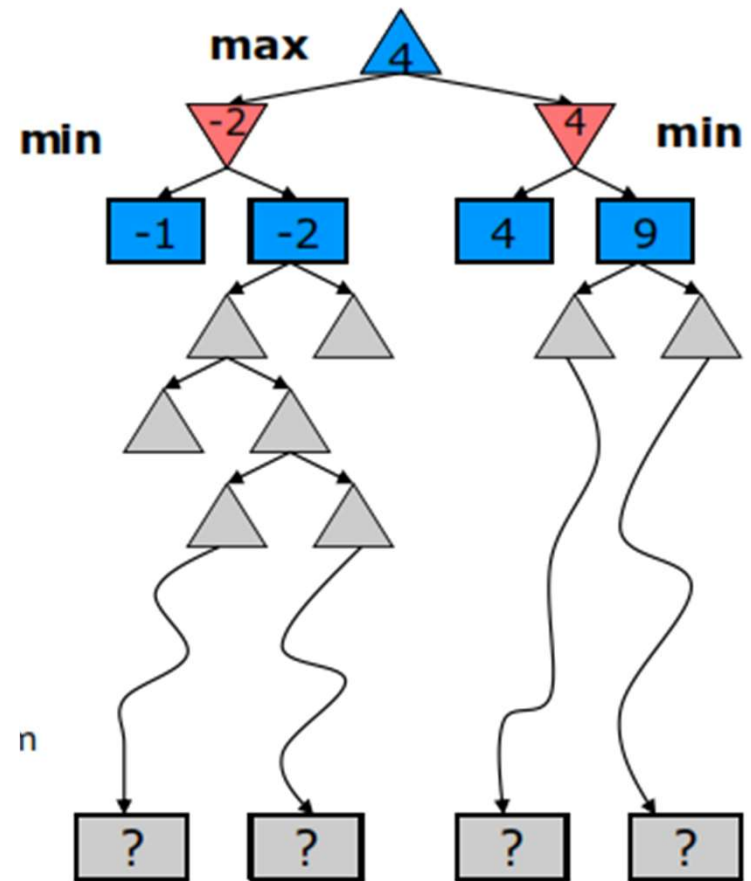
Properties

- Performs a complete depth-first exploration of the tree
- Optimal against a perfect player.
- Time complexity?
 - $O(b^m)$
- Space complexity?
 - $O(bm)$
- For chess, $b \sim 35$, $m \sim 100$
 - Exact solution is completely infeasible
 - But, do we need to explore the whole tree?
- Minimax serves as the basis for the mathematical analysis of games and for more practical algorithms



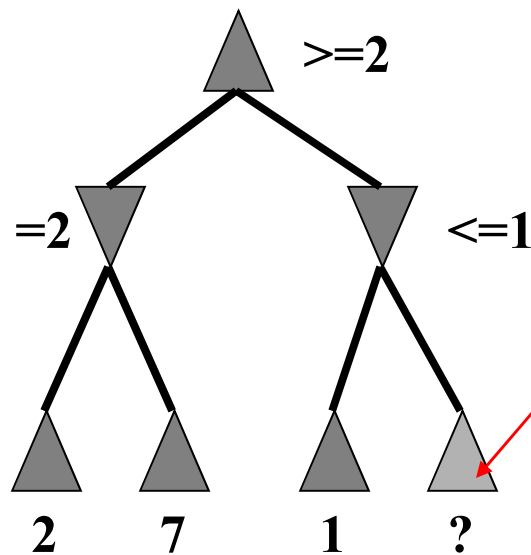
Resource Limits

- Cannot search to leaves
- Depth-limited search
 - Instead, search a limited depth of tree
 - Replace terminal utilities with an eval function for non-terminal positions
- Guarantee of optimal play is gone
- More plies makes a BIG difference
- Example:
 - Suppose we have 100 seconds, can explore 10K nodes / sec
 - So can check 1M nodes per move
 - α - β reaches about depth 8 – decent chess program



Alpha-beta pruning

- We can improve on the performance of the minimax algorithm through alpha-beta pruning.
- Basic idea: *“If you have an idea that is surely bad, don't take the time to see how truly terrible it is.”* -- Pat Winston



- We don't need to compute the value at this node.
- No matter what it is it can't effect the value of the root node.

Parameters of the Alpha Beta Pruning Algorithm

Alpha(α):

The best choice(highest utility/value) found till the current state on the path traversed by the maximizer.

Alpha represents the **minimum score that the maximizing player is assured of**

Beta(β):

The best choice(lowest utility/value) found till the current state on the path traversed by the minimizer.

Beta represents the **maximum score that the minimizing player is assured of**

The critical points of Alpha Beta Pruning in AI are as follows.

The initialization of the parameters

Alpha(α) is initialized with $-\infty$.

Beta(β) is initialized with $+\infty$.

Updating the parameters

Alpha(α) is updated only by the maximizer at its turn.

Beta(β) is updated only by the minimizer at its turn.

Passing the parameters

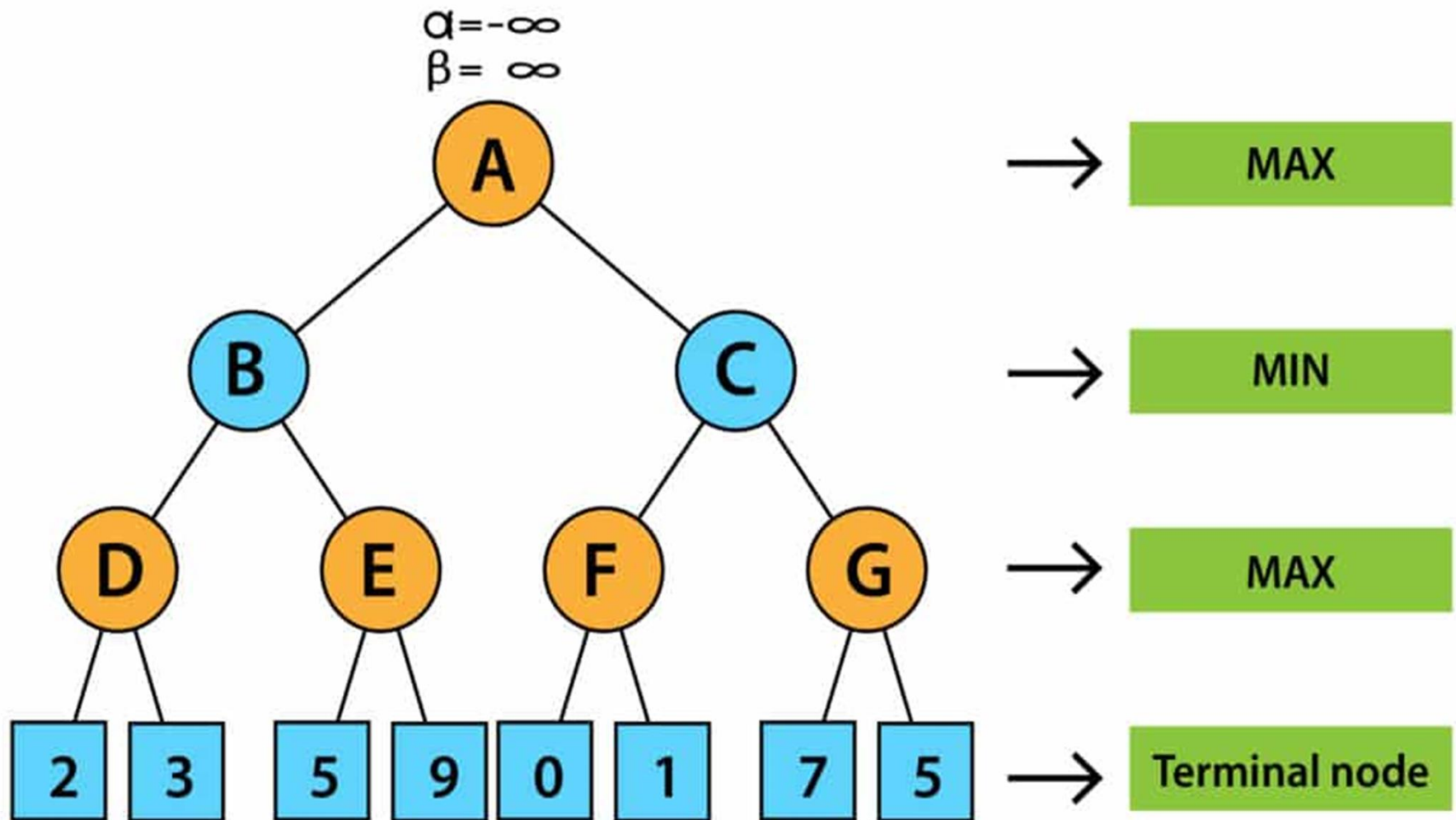
Alpha(α) and Beta(β) are passed on to only child nodes.

While backtracking the game tree, the node values are passed to parent nodes.

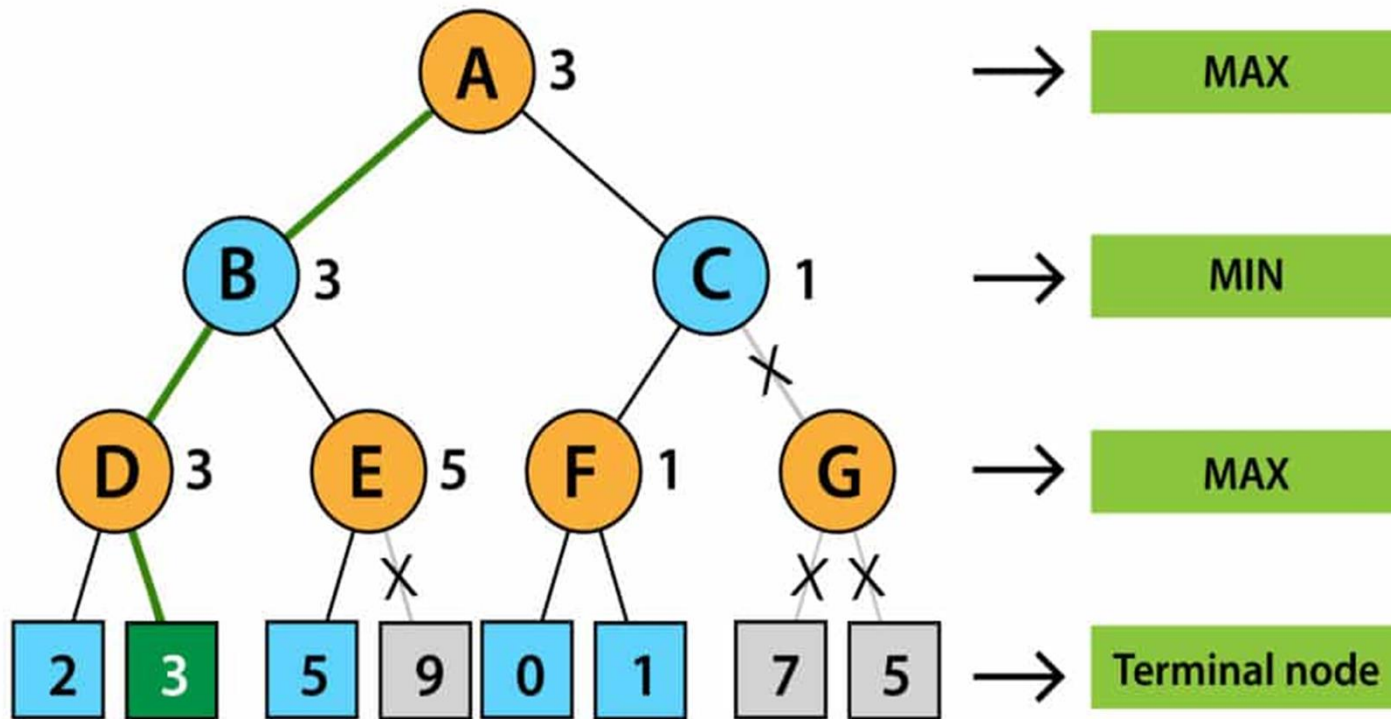
Pruning Condition

The child sub-trees, which are not yet traversed, are pruned if the condition $\alpha \geq \beta$ holds.

Example



Answer



Reference: <https://www.mygreatlearning.com/blog/alpha-beta-pruning-in-ai/>


```
function alpha_beta_pruning(node, alpha, beta, depth, is_maximiser):  
    if the node is a terminal or depth == 0:  
        return utility of node  
  
    if is_maximiser == True:  
        max_utility = -INF  
  
    for each child_node of node:  
        utility = alpha_beta_pruning(child_node, alpha, beta, depth + 1, FALSE)  
        max_utility = max(utility, max_utility)  
        alpha = max(utility, alpha)  
  
        if alpha >= beta:  
            break  
  
    return max_utility
```

```

else:
    min_utility = INF

    for each child_node of node:
        utility = alpha_beta_pruning(child_node, alpha, beta, depth + 1.
TRUE)
        min_utility = min(utility, min_utility)
        beta = min(utility, beta)

    if alpha >= beta:
        break

return min_utility

```

Pseudocode of Alpha-Beta Algorithm

Alpha-beta algorithm in simple words

- Two functions recursively call each other

```
function MAX-value (n, alpha, beta)
  if n is a leaf node then return f(n);
  for each child n' of n do
    alpha := max {alpha, MIN-value(n', alpha, beta)};
    if alpha >= beta then return beta /* pruning */
  end{do}
  return alpha
```

```
function MIN-value (n, alpha, beta)
  if n is a leaf node then return f(n);
  for each child n' of n do
    beta := min {beta, MAX-value(n', alpha, beta)};
    if beta <= alpha then return alpha /* pruning */
  end{do}
  return beta
```

initiation :

$\alpha := -\infty$

$\beta := \infty$

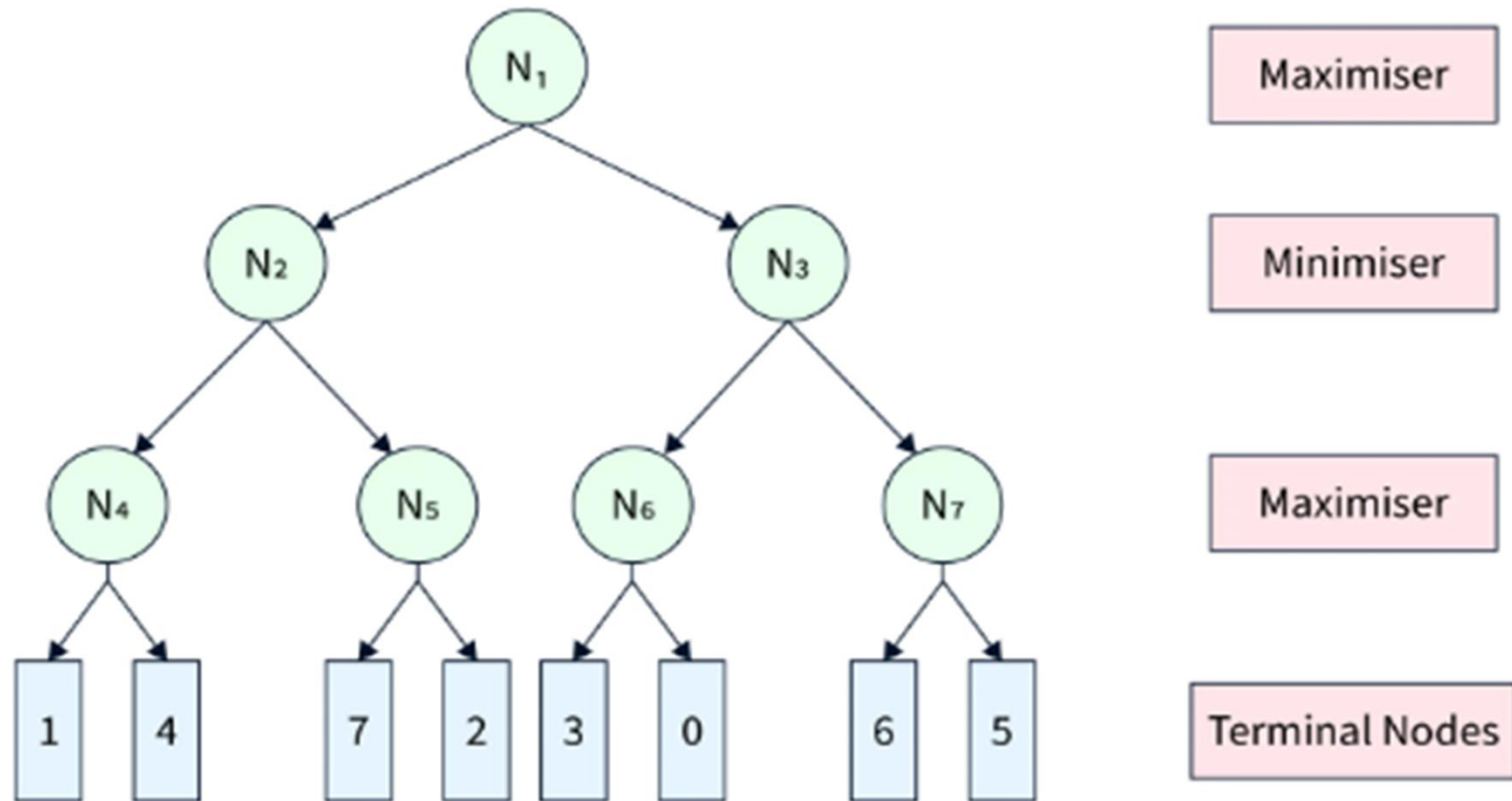
Alpha-beta pruning

- Traverse the search tree in depth-first order
- At each **Max** node n , **alpha(n)** = maximum value found so far
 - Start with **-infinity** and only increase
 - Increases if a child of n returns a value greater than the current alpha
 - Serve as a tentative lower bound of the final pay-off
- At each **Min** node n , **beta(n)** = minimum value found so far
 - Start with **infinity** and only decrease
 - Decreases if a child of n returns a value less than the current beta
 - Serve as a tentative upper bound of the final pay-off

Alpha-beta pruning

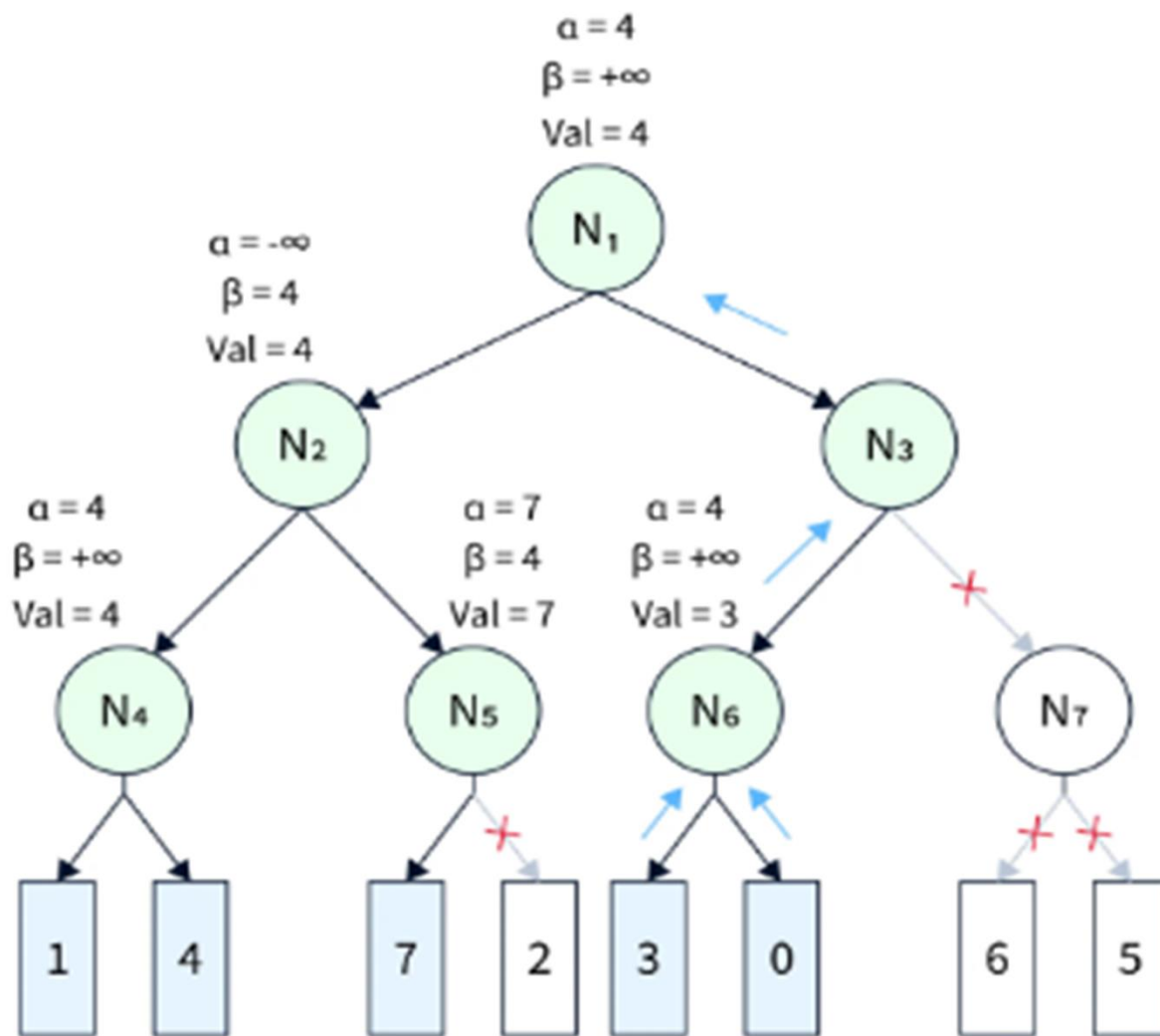
- **Alpha cutoff:** Given a Max node n , cutoff the search below n (i.e., don't generate or examine any more of n 's children) if $\alpha(n) \geq \beta(n)$
(alpha increases and passes beta from below)
- **Beta cutoff.:** Given a Min node n , cutoff the search below n (i.e., don't generate or examine any more of n 's children) if $\beta(n) \leq \alpha(n)$
(beta decreases and passes alpha from above)
- Carry alpha and beta values down during search
Pruning occurs whenever $\alpha \geq \beta$

Example



Reference: <https://www.scaler.com/topics/artificial-intelligence-tutorial/alpha-beta-pruning/>

Solution



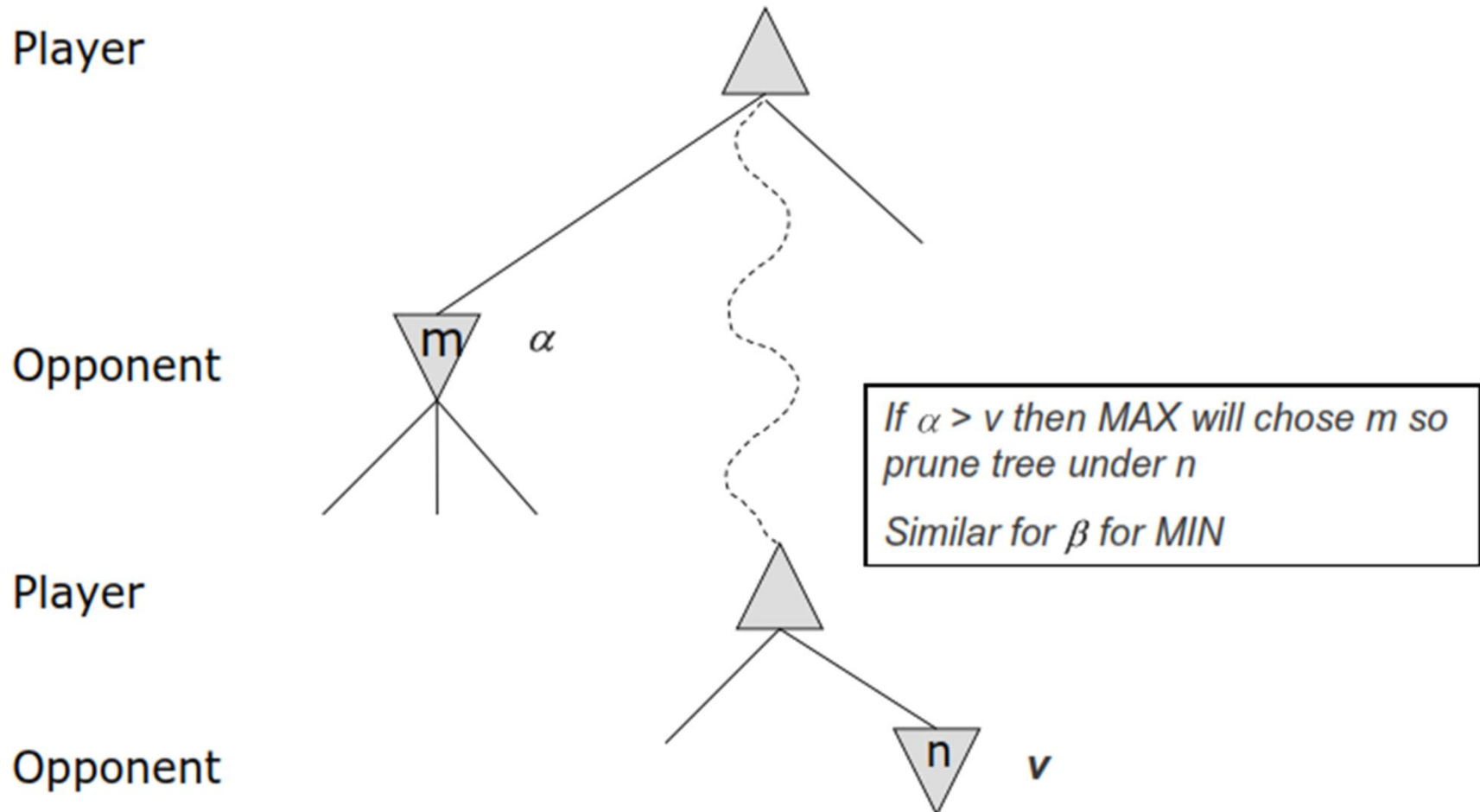
Maximiser

Minimiser

Maximiser

Terminal Nodes

α - β pruning: General Principle



Why is it called α - β ?

- α is the value of the best (i.e., highest-value) choice found so far at any choice point along the path for *max*
 -
 - If v is worse than α , *max* will avoid it
 - - prune that branch
- Define β similarly for *min*

MAX

MIN

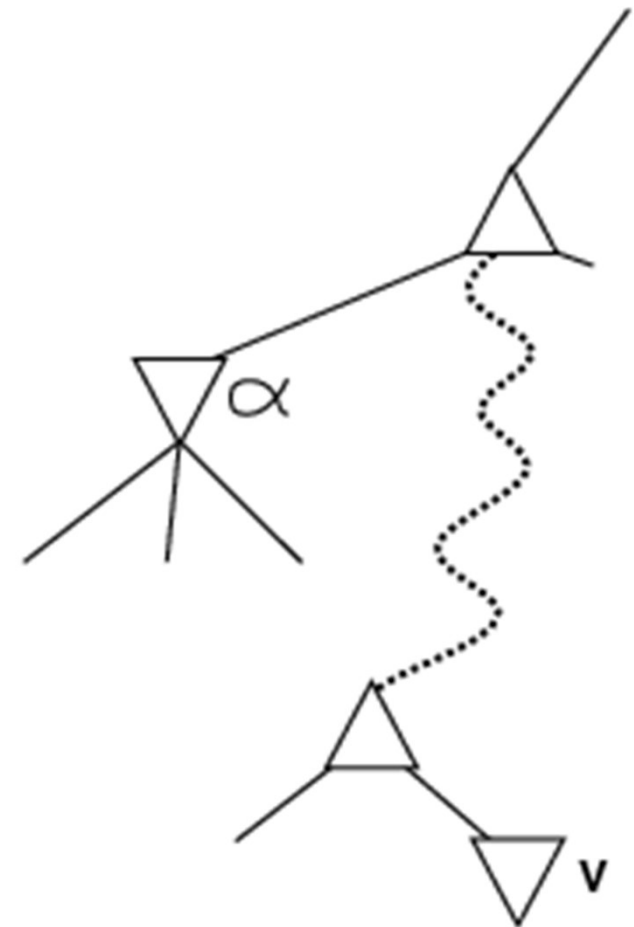
..

..

..

MAX

MIN



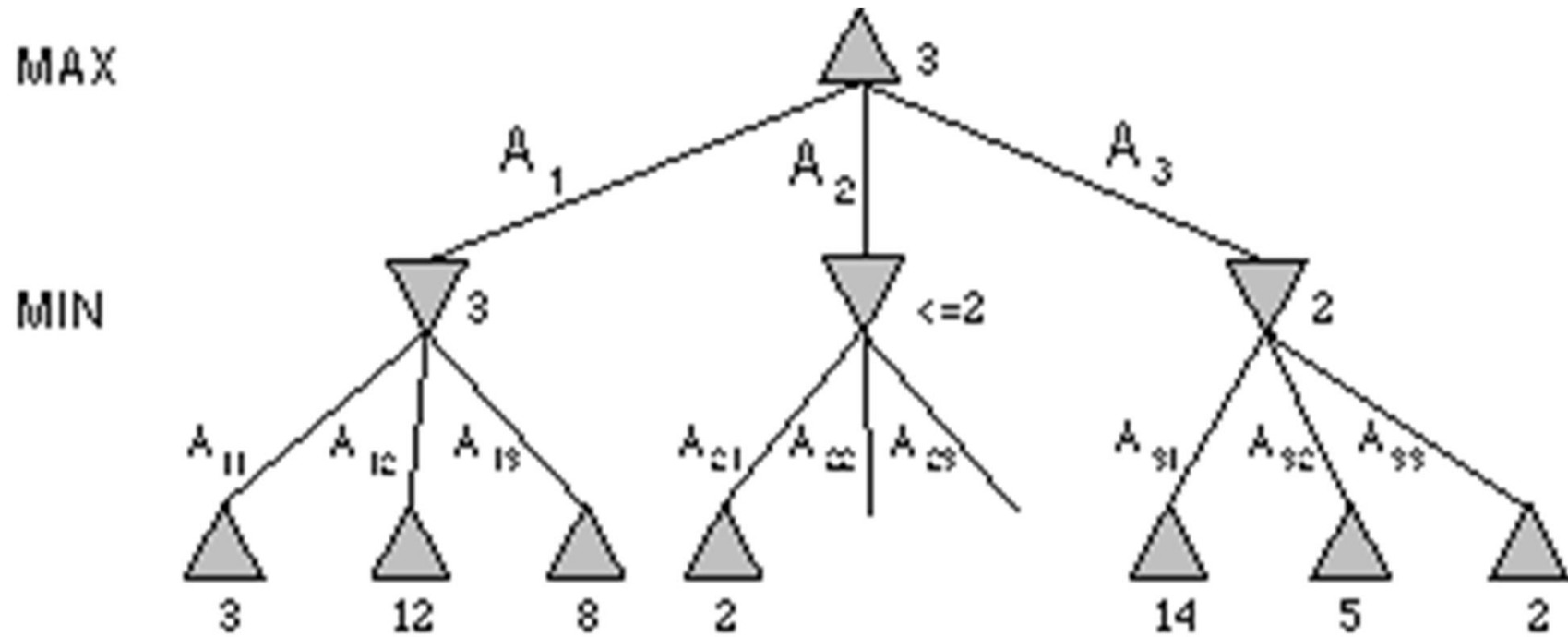
α - β pruning

- Alpha-beta search updates the values of α and β as it goes along and prunes the remaining branches at a node as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN, respectively.
- The effectiveness of alpha-beta pruning is highly dependent on the order in which the successors are examined.

Properties of α - β

- Pruning **does not** affect final result
- Good move ordering improves effectiveness of pruning
- With "perfect ordering," time complexity = $O(b^{m/2})$
→ **doubles** depth of search
- A simple example of the value of reasoning about which computations are relevant (a form of **metareasoning**)

Alpha-beta search



Alpha-beta algorithm

- Two functions recursively call each other

```
function MAX-value (n, alpha, beta)
  if n is a leaf node then return f(n);
  for each child n' of n do
    alpha := max {alpha, MIN-value(n', alpha, beta)};
    if alpha >= beta then return beta /* pruning */
  end{do}
  return alpha
```

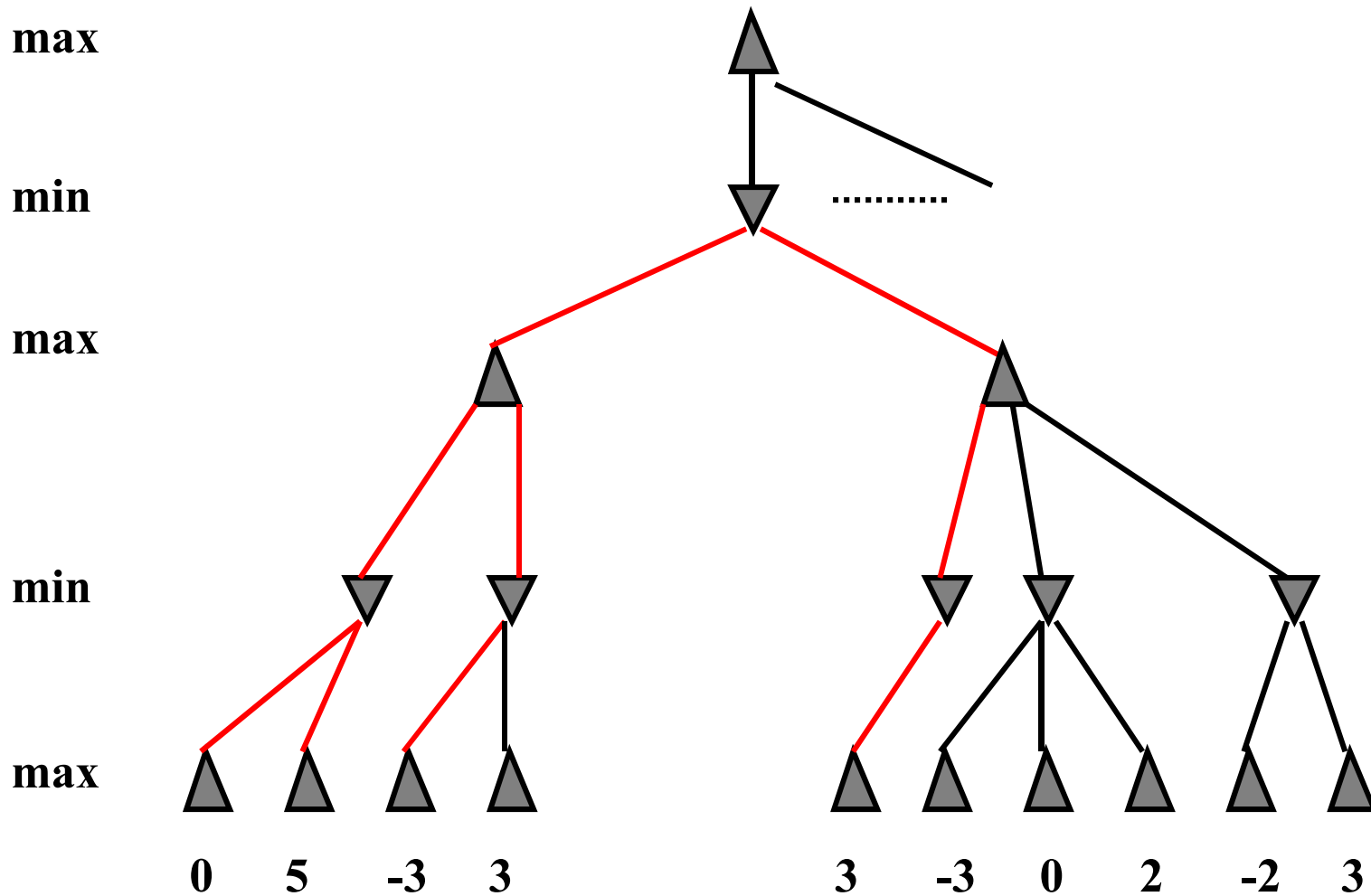
```
function MIN-value (n, alpha, beta)
  if n is a leaf node then return f(n);
  for each child n' of n do
    beta := min {beta, MAX-value(n', alpha, beta)};
    if beta <= alpha then return alpha /* pruning */
  end{do}
  return beta
```

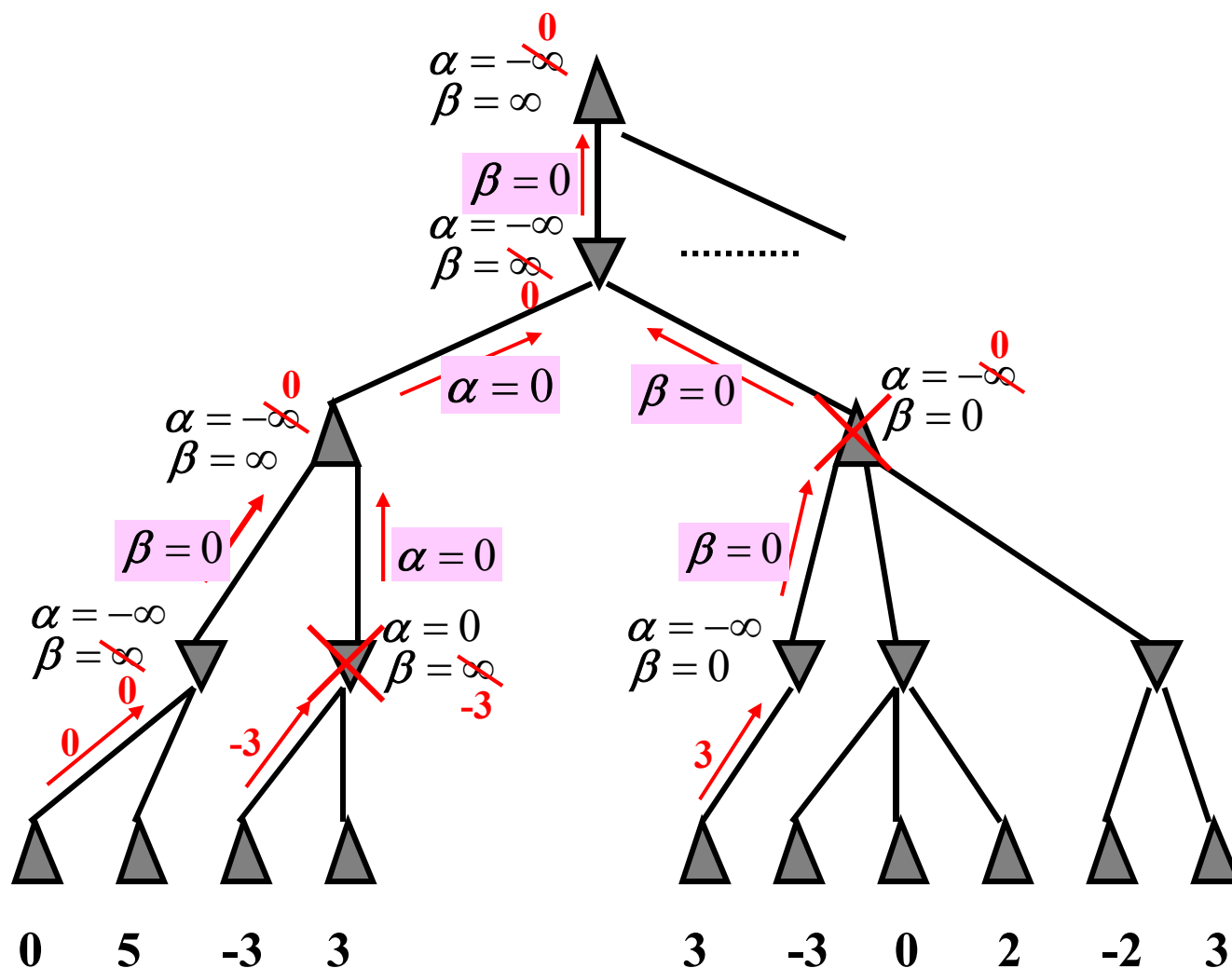
initiation :

$\alpha := -\infty$

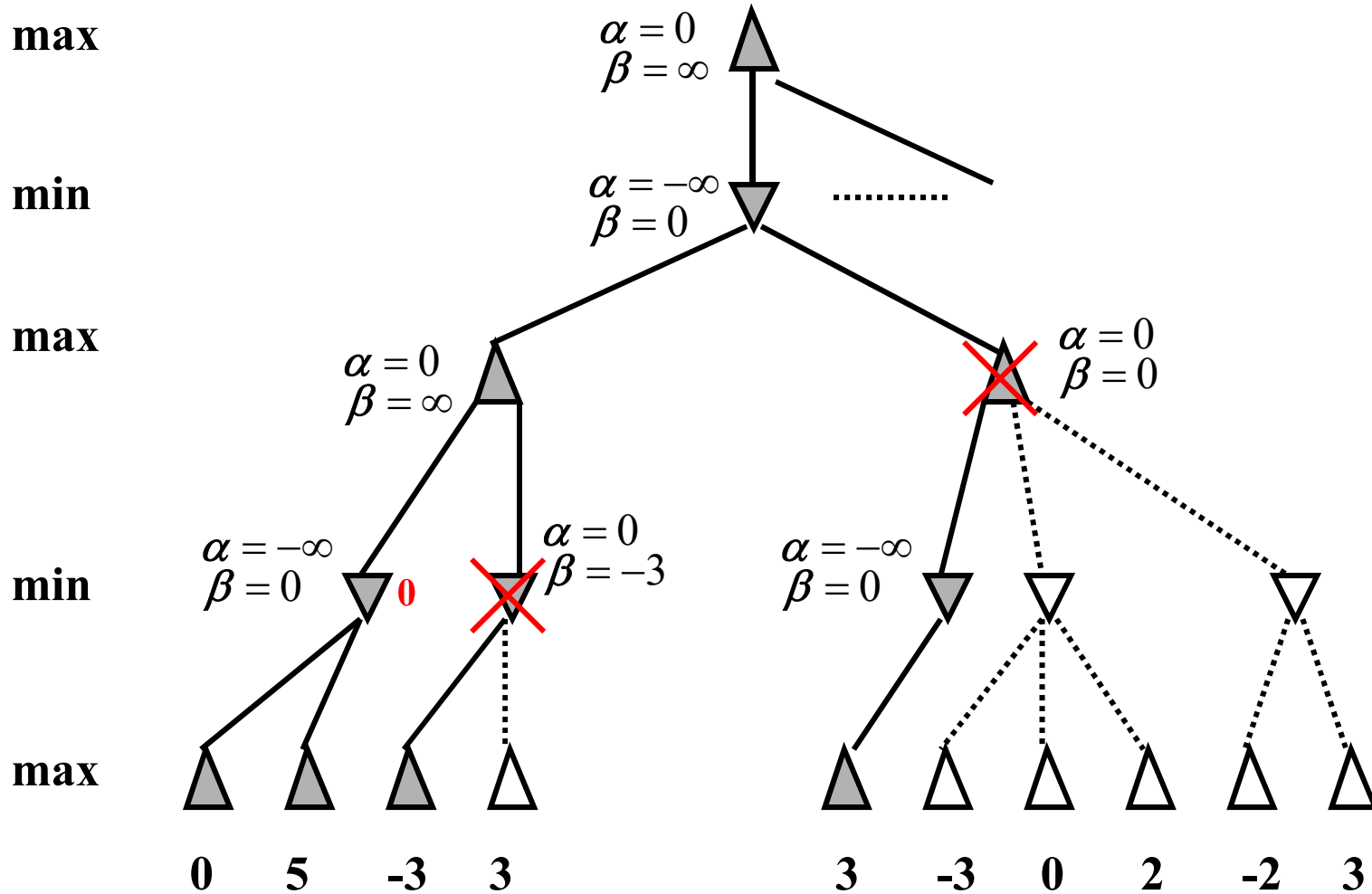
$\beta := \infty$

An example of Alpha-beta pruning



max

Final tree



Effectiveness of Alpha-beta pruning

- Alpha-Beta is guaranteed to compute the same value for the root node as computed by Minimax.
- **Worst case:** NO pruning, examining $O(b^d)$ leaf nodes, where each node has b children and a d -ply search is performed
- **Best case:** examine only $O(b^{(d/2)})$ leaf nodes.
 - Can search twice as deep as Minimax in the same amount of time! Or the branch factor is $b^{(1/2)}$ rather than b .
- **Best case** is when each player's best move is the leftmost alternative, i.e. at MAX nodes the child with the largest value generated first, and at MIN nodes the child with the smallest value generated first.
- In Deep Blue, they found empirically that Alpha-Beta pruning meant that the average branching factor at each node was about 6 instead of about 35-40